System Design Analysis and Proposed Solution

High-Traffic Flash-Sale E-Commerce Architecture

Name: Rohit K Manoj
Email: rohit23110492@snuchennai.edu.in

Case Scenario

Your company is launching an urgent flash-sale feature for an e-commerce platform during a major festival. A limited batch of 500 high-demand smartphones will go live at exactly 8:00 PM. Based on marketing analytics, the platform expects a massive traffic spike of over 100,000 concurrent user requests hitting the checkout endpoint in the first 10 seconds of the sale launch.

Problem Statement

During previous trial runs, the platform suffered from high latency, database row locks, and severe API request timeouts. Worse, a critical consistency bug occurred: the system oversold the inventory, resulting in 540 successful checkouts despite only having 500 physical devices in stock. As a Full-Stack Candidate, you are tasked with re-architecting both the client-facing interface and the backend processing pipeline to ensure absolute data consistency, zero overselling, and a smooth user experience under high load.

Requirement Analysis

Functional Requirements

- View items and status of flash sale items (limited batch of 500 smartphones).
- Users can initiate checkouts for flash sale items, at exactly 8:00.
- User receives real-time order status updates
- User product reviews, ratings, sharing.
- General e-commerce product catalogue search and navigation

Non-Functional Requirements

- Scalability:
    - Handle 100,000 concurrent user requests hitting in the first 10 seconds.
    - Distribute load across backend replicas
- Performance:
    - Frontend/Client Bundle Size < 200KB.
    - Client Time-To-Interactive < 1.5s.
    - Decouple heavy database writes from HTTP request-response cycles.
- Availability: 0% dropped user requests.
- Reliability: ACID compliance, Atomic Decrements.
- Consistency: Guarantee consistent inventory management, exactly 500 items. Zero overselling. Avoid application-level double counters and row lock timeouts.

Solution

Frontend Strategy (Client Optimisation)

Bundle Optimisation

1. Frontend built using lightweight SPA frameworks to maintain bundle footprint below 200KB and TTI less than 1.5s
2. Minification & Compression: Strips whitespaces, variable name mangling, asset compression to reduce bundle size
3. **Tree Shaking & ES6**
    - Use of ES Modules to allow bundlers (Webpack, Rollup, etc) to analyse required and unused code.
    - Bundlers use ES Module static imports to remove dead/unused code, minimising sizes of heavy utility libraries, this is called tree shaking
4. **Code Splitting & Lazy Loading**
    - Code Splitting: Break monolithic code into smaller logical chunks to enable dynamic module loading. For multipage applications use route-based code splitting.
    - Use lazy loading libraries to load modules on demand. Improves Total Blocking Time (total time the main thread was block by JS tasks)

Asset Hosting (CDN)

1. Static Assets such as JS bundles, stylesheets, thumbnails, etc., are distributed globally using Content Delivery Networks.
2. CDNs store copies of these assets in regional edge servers.
3. It directs requests to the nearest edge server via DNS (Proximity Routing).
4. The assets are cached, i.e., files available in the edge server are served immediately (cache hit). If not, they are fetched from the origin server and saved in the edge server (cache miss).
5. Usage of CDNs significantly improve Time-To-Interactive times and reduce/distribute overall server load.

UI Optimisations

1. Debouncing: Client disables action button (Checkout) immediately upon first click and updates UI to indicate request processing status.
2. Duplicate clicks are rejected at the client, preventing repetitive API requests caused by user spamming clicks.
3. Ongoing checkout request stored with unique token to ensure it is acknowledged on subsequent visits.

Status Tracking

1. Dynamic Polling: Client requests server for status updates at exponentially increasing intervals.
2. Web Sockets: Bidirectional persistent connection to exchange status messages.
3. **Architectural Strategy (Server-Sent Events):**
    a. Using Server-Sent Event (SSE), combining advantages of both dynamic polling and web sockets.

b. Dynamic Polling uses standard HTTP protocol for each request incurring high overheads for simple status updates.
c. Web Sockets involves high overheads for session management. The complexity of a full duplex connection is redundant.
d. Server-Sent Events (SSE): A unidirectional (server pushes) open HTTP stream, persistent and incurs minimal overhead while having native reconnection.

Optimistic UI

1. With real-time status updates from server using SSE, the UI can optimistically track order status by assuming next state (Checkout => Order in Queue => Order Success/Failed)
2. On checkout button click, instead of an unresponsive UI waiting for update, the client assumes the next state "Order in Queue", before the acknowledgement of a successful order placement.
3. On receiving update, the UI can either change to the next pre-emptive state "Order Success" (Success) or gracefully rollback with an error message (Failure).

Backend & Middleware Architecture

Ingress & Traffic Distribution (Clustering & Load Balancing):

1. Load Balancer distributes incoming traffic across multiple backend servers.
2. Enables scaling, by allowing backend instance management, i.e., maintenance, fault recovery, horizontally scaling instances to accommodate 100,000 concurrent users during flash sales.
3. Clustering: Connected collection of multiple independent isolated devices (node), acting as a single unified compute system.
4. Load Balancer distributes network traffic to suitable servers in the cluster, also handling health checks

Atomic Reservation (Redis Caching):

1. Traditional databases handle multiple requests concurrently causing a race condition, two requests at the same time, successfully placing an order for the same item, leading to an oversell.
2. However, Redis handles commands sequentially in a single-threaded event loop, hence incoming requests are pushed into a queue processed one by one, preventing race conditions.
3. Redis supports scripting, locking the single thread on execution, making inventory checks and decrementing stock atomic per request (impossible for more than one data modification to happen at the exact same time).
4. On request,
    a. With stock available, the stock is decremented and a 5-minute expiration token using Redis' in-built Time-To-Live support. Returns Acknowledgement and sends event to asynchronous messaging queue.
    b. With empty stock, it immediately sends failure response back.
5. The expiration token ensures payment abandonment, i.e., user drops within 5 minutes.
6. Redis handles stock consistency over high traffic, protecting database from traffic spikes.

Asynchronous Message Queue (Decoupling & Throttling):

1. The asynchronous message broker acts as a buffer, a FIFO queue, enabling syncing services with varying response times.
2. Decoupling: the queue separates fast Redis reservations events from slow database writes.
3. Throttling: It buffers the spike of 100,000 users' requests into a steady stream that the relational database can handle.
4. Here, the Redis order placements are queued, and the database appends records onto an immutable log like a ledger or WAL. To maintain ACID compliance.
5. The Queue retains messages until an ACK is received; hence the data is safe on database crashes.

Data Component (Database Consistency & Concurrency Control)

Message Consumption & Batching

1. Asynchronous workers are configured to fetch a fixed number of messages, a batch. The message queue broker will not assign additional messages unless the batch is acknowledged (ACK)
2. This enables throttling to prevent queue dumping messages into workers at once.
3. Throttling and proper batch size ensures maximum database write throughput without affecting performance.

Conditional SQL Execution

1. Atomic Decrement: on receiving message, workers run conditional Update query if stock not empty.
2. Although this is concurrent, it is executed at a controlled rate, and the update query takes insignificant run time compared to heavy database writes directly without caching and queueing.

Ledger & Transactions

1. On successful, the update query results in a Rows Affected = 1, the worker Inserts the order details and token into an immutable, append only table.
2. The worker task, from Update to ledger Insert is enclosed in a transaction and commit to ensure ACID compliance per batch, improving performance compared to per request transactions.

Acknowledgement & Status Propagation:

1. On successful execution of batch, the worker sends ACK, thus clearing batch’s messages from queue
2. The worker sends a Successful Order Reservation event back to client (though SSE or Redis Pub/Sub)
3. The client on receiving the event, updates UI with success message and order tracking.

Conclusion

Feature	Legacy System	Proposed Solution
Request Flow	Synchronous, blocking database calls.	Asynchronous, decoupled queue processing.
Stock Validation	Soley based on Relational DB Atomicity.	In-Memory Redis + Atomic Scripting + Atomic DB Queries.
Overselling Vulnerability	Highly vulnerable, concurrent DB operations lead to race condition.	Atomic Redis Reservation + Conditional SQL. Secure against overselling.
Peak Throughput Limit	Restricted by database lock bottlenecks.	High scalability (Gateway & Redis tier)
User Experience	Blocking page render, duplicate clicks, poor status updates.	Optimised load times, debounced UI, optimistic SSE status tracking.
